

Algoritmos Genéticos

Busca por uma população de soluções

Seleção, recombinação e mutação para explorar espaços de busca.



Prof. Dr. João Choma

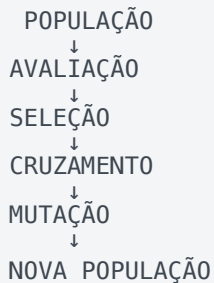
Universidade Estadual de Maringá — UEM

github.com/JoaoChoma

Algoritmos de Busca e Otimização



Algoritmos Genéticos são inspirados em conceitos da evolução:





Soluções com melhor desempenho têm maior possibilidade de contribuir para a próxima geração.



O que estamos buscando?

Em problemas de otimização, normalmente não buscamos apenas:

"um estado objetivo"

Por exemplo:

- menor custo;
- maior cobertura;
- melhor distribuição;

Buscamos:

a melhor solução possível
segundo uma função de qualidade

- menor tempo;
- maior desempenho;
- menor número de conflitos.



Considere um problema com muitas configurações possíveis:





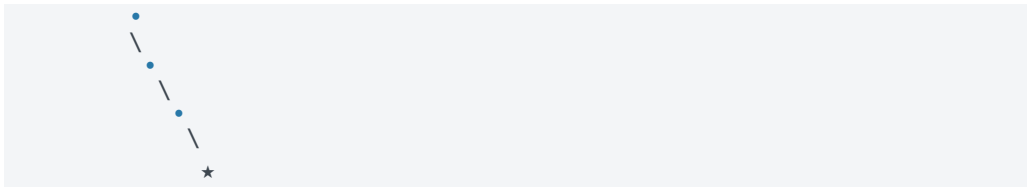
Cada • representa uma solução candidata.

★ representa uma solução ótima ou de alta qualidade.

O desafio é encontrar boas regiões sem avaliar todas as possibilidades.



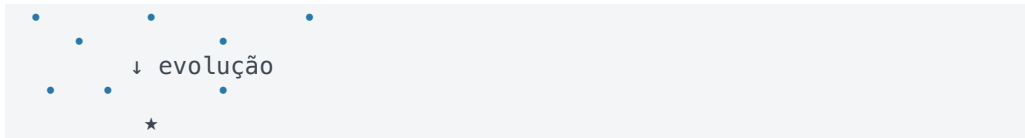
Hill Climbing



Explora a partir de **um ponto**.



Algoritmo Genético





Explora o espaço utilizando **uma população de pontos**.



Cada solução candidata é chamada de:

INDIVÍDUO

Por exemplo, imagine uma ordem de execução de casos de teste:

[TC1, TC2, TC3, TC4, TC5]

Outro indivíduo:

[TC3, TC1, TC5, TC2, TC4]

Cada indivíduo representa uma possível solução do problema.



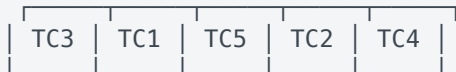
A representação de um indivíduo costuma ser chamada de:

CROMOSSOMO

Exemplo:

[TC3, TC1, TC5, TC2, TC4]

Cada posição ou componente pode ser tratada como um **gene**.



↑
gene



Antes de executar o algoritmo precisamos decidir:

Como uma solução será codificada?

Exemplos:

Problema de seleção:
[1, 0, 1, 1, 0]

Problema de ordenação:
[TC3, TC1, TC5, TC2, TC4]



Problema de parâmetros:
[0.45, 12, 0.82, 7]

Uma representação inadequada pode tornar a busca ineficiente ou gerar soluções inválidas.



Considere um cromossomo binário com 4 genes:

`[0,0,0,0]`

Cada gene pode assumir:

0 ou 1

Então o espaço contém:
soluções possíveis.

0000	0001	0010	0011
0100	0101	0110	0111
1000	1001	1010	1011
1100	1101	1110	1111

$$2^4 = 16$$

O AG procura boas soluções dentro desse espaço.



Com 10 genes binários:

$$2^{10} = 1.024$$

Com 30:

$$2^{30} = 1.073.741.824$$

Com 100:

$$2^{100}$$

Avaliar todas as soluções pode se tornar impraticável.

Algoritmos Genéticos tentam realizar uma **busca aproximada e orientada** nesse espaço.



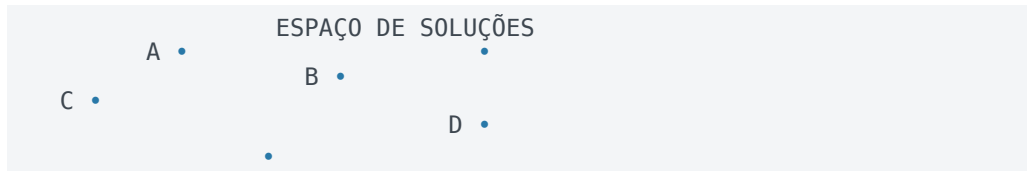
Em vez de começar com uma solução:

Solução atual

Cada indivíduo ocupa um ponto diferente do espaço de soluções.

começamos com várias:

POPULAÇÃO INICIAL
A = [1,0,0,1,0]
B = [1,1,0,0,1]
C = [0,1,1,0,0]
D = [1,0,1,1,0]





A população distribui vários pontos pelo espaço.

Depois, o algoritmo tenta deslocar as próximas gerações para regiões de maior qualidade.



Terceiro elemento — função de fitness

Precisamos avaliar cada indivíduo.

A função de fitness responde:

Quão boa é esta solução?

Se estamos maximizando:

$D > B > A > C$

Definimos:

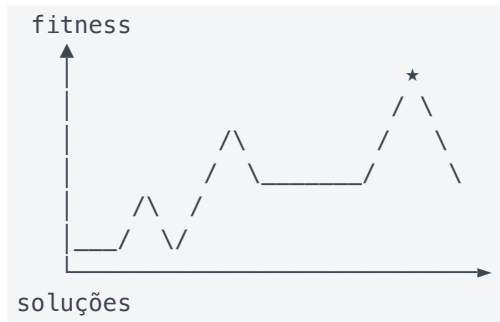
`fitness(indivíduo)`

Exemplo:

A → fitness = 42
B → fitness = 71
C → fitness = 35
D → fitness = 83



Podemos imaginar uma paisagem:



Cada solução possui uma qualidade.

O objetivo do AG é encontrar regiões com valores elevados de fitness.



A população pode ser criada aleatoriamente:

```
Geração 0
A = 01001    fitness = 30
B = 11100    fitness = 65
C = 00111    fitness = 42
D = 10110    fitness = 80
E = 01101    fitness = 55
```

Neste momento, estamos espalhando pontos pelo espaço.



Agora precisamos decidir:

Quais indivíduos terão maior chance de gerar descendentes?

Em geral:

melhor fitness
↓
maior chance de seleção

Mas isso não significa necessariamente:

somente o melhor sobrevive

Manter diversidade é importante.



Temos:

```
A → 30  
B → 65  
C → 42  
D → 80  
E → 55
```

Uma seleção poderia escolher:

D e B

como pais.

```
D = 10110  
B = 11100
```

Agora podemos combinar características das duas soluções.



Escolhemos um ponto de corte:

```
Pai 1:  
101 | 10  
Pai 2:  
111 | 00
```




Trocamos partes:

```
Filho 1:  
101 | 00  
=  
10100
```



```
Filho 2:  
111 | 10  
=  
11110
```

Novas regiões do espaço podem ser alcançadas pela combinação de soluções existentes.



Considere:

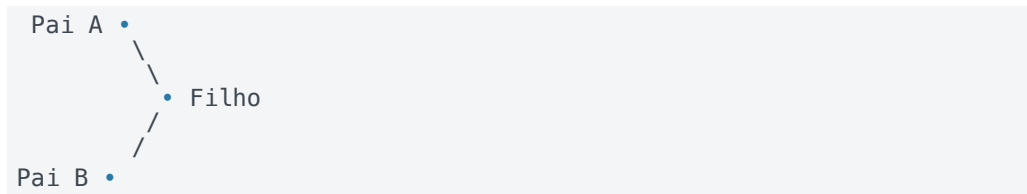
Pai A



Pai B



O cruzamento tenta combinar características dos pais:



Não significa geometricamente que o filho estará "entre" os pais.



A posição depende da representação e do operador de crossover.



Depois do crossover, podemos realizar pequenas alterações aleatórias.

Mutação no terceiro gene:

10000

A mutação introduz novas características na população.

Antes:

10100

ou:

10100

↓

10110



Imagine que toda a população tenha:

```
gene 3 = 0
```

Sem mutação, talvez nunca apareça:

```
gene 3 = 1
```

A mutação permite explorar regiões que poderiam não ser alcançadas apenas pelo crossover.



O AG precisa equilibrar duas ideias.

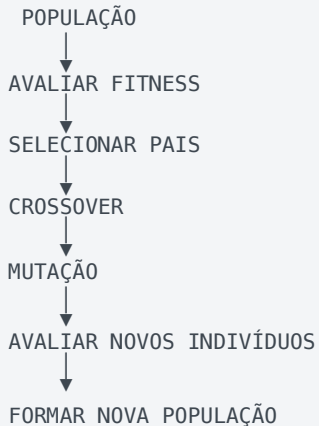
Aproveitamento — exploitation

Usar boas soluções existentes para gerar novas soluções.

Exploração — exploration

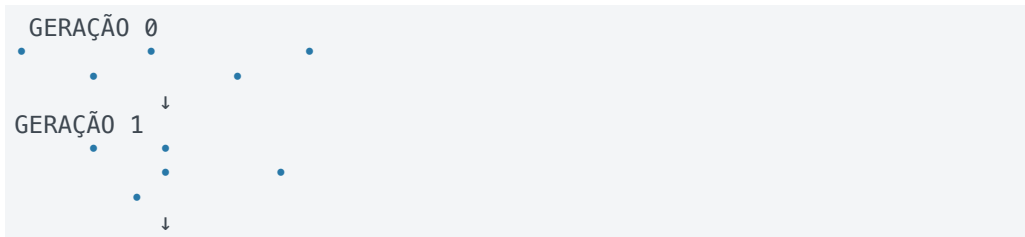
Visitar novas regiões do espaço de busca.

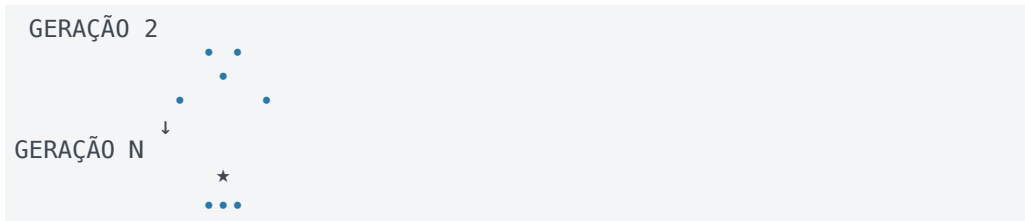
Seleção e crossover favorecem o aproveitamento.
Mutações ajudam a manter exploração e diversidade.





Esse processo corresponde a uma **geração**.







Idealmente, a população começa a se concentrar em regiões melhores.



```
populacao = gerar_populacao()
while not criterio_parada():
    avaliar(populacao)
    nova_populacao = []
    while len(nova_populacao) < tamanho_populacao:
        pai1 = selecionar(populacao)
        pai2 = selecionar(populacao)
        filho1, filho2 = crossover(pai1, pai2)
        filho1 = mutar(filho1)
        filho2 = mutar(filho2)
        nova_populacao.extend([filho1, filho2])
    populacao = nova_populacao
return melhor_individuo(populacao)
```



O que substitui a fronteira?

Na BFS e DFS falávamos em:

FRONTEIRA

No Hill Climbing:

ESTADO ATUAL + VIZINHANÇA

No Algoritmo Genético:

POPULAÇÃO

A população é o conjunto de soluções que o algoritmo mantém ativamente durante a busca.



BFS

Expande sistematicamente vários estados por níveis.

DFS

Aprofunda um caminho antes de retornar.

Hill Climbing

Mantém uma solução e escolhe melhorias locais.

Algoritmo Genético

Mantém várias soluções e faz a população evoluir.



BFS





Hill Climbing

S • → • → • → ★



Algoritmo Genético

G0: • • • •



G1: • • •



G2: • • ★



Essa diferença é importante.
é possível interpretar a solução como um caminho.

Em um AG:

```
Indivíduo A  
Indivíduo B  
Indivíduo C  
...
```

Na BFS:

$S \rightarrow A \rightarrow D \rightarrow G$

cada indivíduo normalmente já representa uma **solução completa candidata**.

O objetivo é melhorar a qualidade das soluções ao longo das gerações.



Imagine que precisamos escolher funcionalidades para uma versão do software.

Uma solução:

```
[1, 0, 1, 1, 0]
```

Temos cinco funcionalidades:

```
F1 F2 F3 F4 F5
```

significa:

```
F1 → selecionada  
F2 → não selecionada  
F3 → selecionada  
F4 → selecionada  
F5 → não selecionada
```



Cada funcionalidade possui:

```
valor para o cliente  
+  
custo de desenvolvimento
```

Exemplo:



Funcionalidade	Valor	Custo
F1	10	5
F2	8	4
F3	15	8
F4	7	3
F5	12	7



Restrição:

orçamento máximo = 15

Objetivo:

Maximizar o valor entregue sem ultrapassar o orçamento.



Indivíduo:

`[1,0,1,0,0]`

Representa:

`F1 + F3`

Custo:

`5 + 8 = 13`

Valor:

`10 + 15 = 25`

Fitness possível:

`fitness = valor total`

desde que:

`custo <= 15`



Com 5 funcionalidades:

$$2^5 = 32$$



possíveis combinações.

```
00000  
00001  
00010  
...  
10100  
...  
11111
```

Para cinco funcionalidades é fácil testar tudo.



Espaço de soluções do exemplo

Mas com 100 funcionalidades:

$$2^{100}$$

O espaço cresce rapidamente.



Podemos começar com:

```
A = 10100  
B = 01011  
C = 11000  
D = 00110
```

Depois calculamos:

```
fitness(A)  
fitness(B)  
fitness(C)  
fitness(D)
```

Algumas soluções podem ser inválidas por ultrapassar o orçamento.



Existem diferentes estratégias.

Uma possibilidade é aplicar uma penalização:

```
if custo > orcamento:  
    fitness = valor - penalidade  
else:  
    fitness = valor
```

Assim, soluções que violam restrições tornam-se menos atraentes.

A função de fitness também pode incorporar regras e restrições do problema.



```
Geração 0
10100
01011
11000
00110
    ↓ seleção
10100
11000
    ↓ crossover
10000
11100
    ↓ mutação
10001
    ↓ avaliação
nova população
```



Evolução no exemplo

O processo continua por várias gerações.



Critério de parada

Quando o algoritmo deve terminar?

Possibilidades:

número máximo de gerações

fitness desejado alcançado

sem melhoria por N gerações

limite de tempo

O AG não sabe necessariamente se encontrou o ótimo global.



Ao final:

```
População final
A → fitness 72
B → fitness 81
C → fitness 76
D → fitness 93  ★
E → fitness 88
```

Retornamos:

```
melhor indivíduo encontrado
```

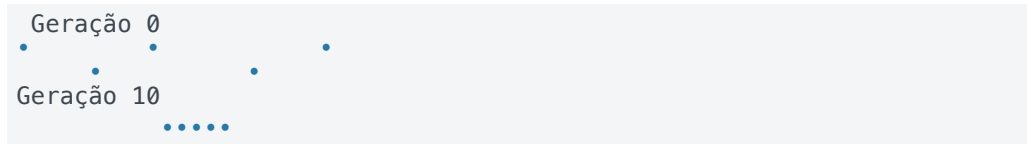
Não necessariamente:

```
melhor indivíduo possível
```

Essa distinção é importante.



Imagine:





A população pode perder diversidade cedo demais.

Todos os indivíduos ficam muito parecidos.

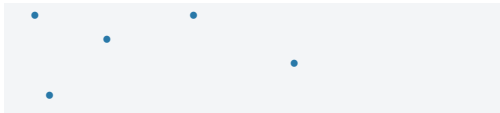
O algoritmo pode se concentrar em uma região:

máximo local

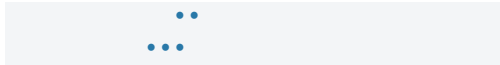
e deixar de explorar regiões melhores.



Uma população diversa representa:



Uma população pouco diversa:



Diversidade ajuda o algoritmo a explorar diferentes regiões do espaço.



Um AG normalmente envolve decisões como:

tamanho da população

taxa de crossover

taxa de mutação



método de seleção

critério de parada

representação do cromossomo

Esses parâmetros influenciam diretamente o comportamento da busca.



O algoritmo depende fortemente de decisões de projeto:

Uma função de fitness inadequada pode fazer o algoritmo otimizar exatamente aquilo que não desejávamos.

REPRESENTAÇÃO
+
FITNESS
+
OPERADORES
+
PARÂMETROS
=
COMPORTAMENTO DA BUSCA



Algoritmos Genéticos podem ser utilizados quando problemas de Engenharia de Software são formulados como otimização.

- geração de dados de teste;
- planejamento de releases;
- modularização;
- alocação de recursos.

Exemplos:

- priorização de casos de teste;
- seleção de requisitos;
- refatoração;
- configuração de sistemas;



Essa ideia leva a uma área importante:

Search-Based Software Engineering

Problemas de Engenharia de Software são reformulados como problemas de busca e otimização.

Precisamos definir:

1. Representação da solução
2. Espaço de soluções
3. Função de fitness
4. Operadores de busca



Depois podemos utilizar técnicas como:

- Hill Climbing
- Simulated Annealing
- Algoritmos Genéticos
- outras meta-heurísticas



Até aqui, tratamos o fitness como um único valor:

fitness = qualidade da solução

- minimizar o custo de desenvolvimento;
- maximizar a cobertura das necessidades dos usuários.

Em otimização multiobjetivo, nem sempre existe uma única solução que seja a melhor em todos os critérios.

Entretanto, muitos problemas reais possuem objetivos conflitantes.

No planejamento de uma release, podemos desejar:

- maximizar o valor entregue;
- minimizar o risco;

Melhorar um objetivo pode piorar outro.



Considere três soluções candidatas:

Queremos:

```
maximizar valor  
minimizar custo
```

Solução	Valor entregue	Custo
A	70	30
B	85	50
C	60	45

A solução B entrega mais valor, mas também custa mais que A.

A solução C entrega menos valor e custa mais que A.



Uma solução domina outra quando:

- 1 não é pior em nenhum objetivo;
- 2 é estritamente melhor em pelo menos um objetivo.

No exemplo:

A domina C

porque A possui:

maior valor: $70 > 60$
menor custo: $30 < 45$

Mas A não domina B, e B não domina A: existe uma troca entre valor e custo.



As soluções que não são dominadas por nenhuma outra formam a:

FRONTEIRA DE PARETO



A fronteira representa diferentes compromissos eficientes entre os objetivos.



Uma fronteira de Pareto não escolhe automaticamente uma única solução.

A escolha final depende das preferências e restrições de quem decide.

Por exemplo:

- uma release estratégica pode aceitar maior custo para entregar mais valor;

O algoritmo encontra compromissos; o contexto determina qual compromisso é mais adequado.

Ela apresenta alternativas eficientes:

menor custo ← → maior valor

- uma equipe com orçamento rígido pode preferir uma solução de menor custo;
- uma organização avessa a falhas pode priorizar menor risco.



Em um Algoritmo Genético multiobjetivo, cada indivíduo pode receber um vetor de avaliação:

```
fitness(indivíduo) = [valor,  
custo, risco]
```

O objetivo é evoluir uma população que represente diferentes regiões da fronteira de Pareto.

Em vez de ordenar toda a população por um único número, podemos considerar:

```
dominância  
+  
diversidade  
+  
proximidade da fronteira desejada
```



Para comparar indivíduos, medir similaridade ou preservar diversidade, podemos calcular a distância entre dois vetores.

$$y = (y_1, y_2, \dots, y_n)$$

Essas distâncias podem ser calculadas no espaço dos genes ou no espaço dos objetivos.

Considere:

$$x = (x_1, x_2, \dots, x_n)$$

Duas métricas comuns são:

distância Euclidiana
distância Manhattan



A distância Euclidiana corresponde à distância em linha reta:

$$d_E(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Exemplo:

$x = (2, 3)$
 $y = (5, 7)$

$$d_E(x, y) = \sqrt{(5 - 2)^2 + (7 - 3)^2} = 5$$

Ela aumenta a influência de diferenças maiores por elevar cada diferença ao quadrado.



A distância Manhattan soma os deslocamentos em cada dimensão:

$$d_M(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Para os mesmos pontos:

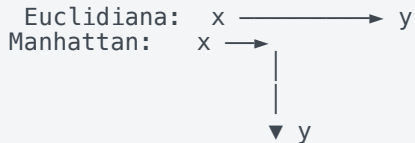
$$\begin{aligned}x &= (2, 3) \\ y &= (5, 7)\end{aligned}$$

$$d_M(x, y) = |5 - 2| + |7 - 3| = 7$$

Ela é chamada Manhattan porque lembra um deslocamento por ruas organizadas em uma grade.



Métrica	Interpretação	Característica
Euclidiana	linha reta	ênfatiza diferenças maiores
Manhattan	soma por dimensão	mede deslocamentos acumulados



A escolha depende da representação e do significado das dimensões do problema.

Uma métrica só é útil quando sua noção de proximidade corresponde ao que significa “ser semelhante” no problema.



Para cromossomos binários, também podemos contar quantos genes são diferentes.

```
A = 1 0 1 1 0
B = 1 1 0 1 0
    ↑  ↑
```

Existem duas posições diferentes.

Essa medida é chamada de:

distância de Hamming = 2

Para vetores binários, a distância Manhattan produz o mesmo resultado que a distância de Hamming.



Se os indivíduos estão muito próximos:

$$d(\text{indivíduo A, indivíduo B}) \approx 0$$

podemos estar explorando regiões distintas.

Aplicações possíveis:

- manter soluções espalhadas pela fronteira de Pareto;
- acompanhar a perda de diversidade entre gerações.

a população pode estar se tornando homogênea.

Se existem distâncias maiores entre diferentes indivíduos:

população mais distribuída

- evitar selecionar pais quase idênticos;
- identificar grupos de soluções semelhantes;



Antes de medir, normalize os objetivos

Considere dois objetivos com escalas diferentes:

custo:	0 a 1.000.000
risco:	0 a 10

Distância Euclidiana ou Manhattan sem atenção às escalas pode produzir uma interpretação enganosa.

Sem normalização, o custo tende a dominar o cálculo da distância.

Uma normalização comum é:

$$x'_i = \frac{x_i - \min_i}{\max_i - \min_i}$$

Depois da normalização, cada objetivo pode ser comparado em uma escala semelhante.



Avaliando uma fronteira aproximada

Em problemas multiobjetivo, podemos avaliar o resultado do AG observando:

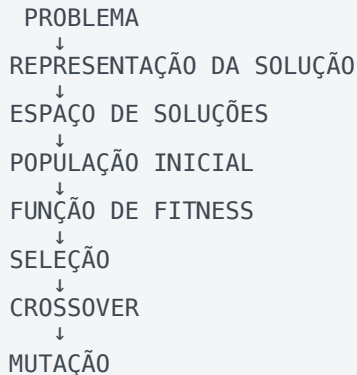
- **diversidade** — existem compromissos diferentes ou todas as soluções são parecidas?
- **dominância** — quantas soluções encontradas são não dominadas?

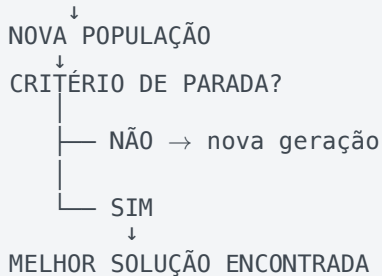
Distâncias Euclidiana, Manhattan ou métricas específicas podem apoiar essas análises.

boa aproximação = proximidade +
diversidade + distribuição

- **convergência** — as soluções estão próximas da melhor fronteira conhecida?
- **distribuição** — as soluções estão bem espalhadas ao longo da fronteira?

Não basta encontrar um único ponto de alta qualidade quando o problema exige alternativas.







BFS / DFS



Como percorrer estados?



procurar um objetivo/caminho



Hill Climbing



Como melhorar uma solução?



uma solução + vizinhança local



Algoritmo Genético



Como evoluir várias soluções?



população + seleção + variação



Algoritmo	Unidade principal	Como explora
BFS	estado/fronteira	níveis
DFS	estado/fronteira	profundidade
Hill Climbing	solução atual	melhor vizinho
Algoritmo Genético	população	evolução de soluções

Essa mudança de perspectiva é fundamental.



Problema:

Escolher funcionalidades para uma release com orçamento limitado.

Defina:

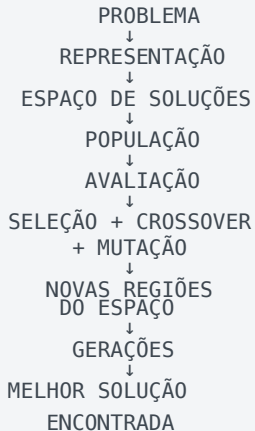
- 1 O que representa um **indivíduo**?
- 2 Como será o **cromossomo**?
- 3 Qual é o **espaço de soluções**?
- 4 Como gerar a **população inicial**?
- 5 Como calcular o **fitness**?
- 6 Como tratar soluções inválidas?
- 7 Como realizar a **seleção**?



- 1 Como realizar o **crossover**?
- 2 Como realizar a **mutação**?
- 3 Qual será o **critério de parada**?
- 4 Existe mais de um **objetivo** relevante?
- 5 Como identificar soluções **dominadas** e a **fronteira de Pareto**?
- 6 Qual métrica de distância representa melhor a **diversidade** da população?



A ideia que deve permanecer





Algoritmos Genéticos exploram o espaço de busca por meio da evolução de uma população de soluções candidatas.